

LAD-6

1. Earliest deadline first.

```
#include <stdio.h>
```

```
struct Process {
```

```
    int id, at, ct, period, deadline, remaining;
```

```
};
```

```
int main() {
```

```
    int n, i, t = 0, completed = 0;
```

```
    printf("Enter number of process: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Enter AT, CPU time, Period & deadline for P%d: ", i+1);
```

```
        scanf("%d %d %d %d", &p[i].at, &p[i].period, &p[i].deadline);
```

```
        p[i].id = i+1;
```

```
        p[i].remaining = p[i].ct;
```

```
    }
```

```
    float v = 0;
```

```
    for (i = 0; i < n; i++) {
```

```
        v += (float) p[i].ct / p[i].period;
```

```
    }
```

```
    if (v > 1)
```

```
        printf("Not feasible\n");
```

```
    else
```

```
        printf("Feasible\n");
```

```
    int gantt[100];
```

```
    while (completed < n) {
```

```
        int idx = -1;
```

```
        int min_deadline = 9999;
```

```
        for (t = 0; t < n; t++) {
```

```
            if (p[i].at <= t && p[i].remaining > 0) {
```

```
                if (p[i].deadline < min_deadline) {
```

```
                    min_deadline = p[i].deadline;
```

```
                    idx = i;
```

```
                }
```

```
            }
```

```

if (id x == -1) {
    gant[i] = p[id].id;
    p[id].remaining--;
    if (p[id].remaining == 0)
        completed++;
} else {
    gant[i] = 0;
}
i++;

```

```

}
printf("\n");
for (i = 0; i < t; i++)
    printf("----+");
printf("\n");
for (i = 0; i < t; i++) {
    if (gant[i] == 0)
        printf("Idle ");
    else
        printf("P %d - %d", gant[i]);
}

```

```

printf("\n");
for (i = 0; i < t; i++)
    printf("----+");
printf("\n");
for (i = 1; i <= t; i++)
    printf(" %d", i);
return 0;
}

```

$$\frac{C_i}{D_i} \leq 1$$

Q.P.

Enter number of process: 3.

Enter AT, CPU Time, Period & deadline for

P1: 0
3
20
7

P2: 0
2
6
4

P3: 0
2
10
8

Feasible

P2	P2	P1	P1	P1	P3	P3
0	1	2	3	4	5	6

Ru
27/4/26

Proportional Share Scheduling

#include <stdio.h>

struct process

{
int id, at, bt, weight, remaining;
};

int main()

{
int n, i, t = 0, completed = 0, tq;

printf("Enter number of processes: ");

scanf("%d", &n);

struct process p[n];

for (i = 0; i < n; i++)

{
printf("Enter At, Bt, weight for p[%d]: ", i+1);

scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].weight);

p[i].id = i+1;

p[i].remaining = p[i].bt;

}

printf("Enter time quantum: ");

scanf("%d", &tq);

int gant[200], g = 0;

while (completed < n)

{
int total_weight = 0;

for (i = 0; i < n; i++)

{
if (p[i].remaining > 0 && p[i].at == t)

{
total_weight += p[i].weight;

}

int executed = 0;

for (i = 0; i < n; i++)

{
if (p[i].remaining > 0 && p[i].at == t)

{
int slice = (p[i].weight * tq) / total_weight;

if (slice < 1)

{
slice = 1;

int k;

for (k = 0; k < slice && p[i].remaining > 0; k++)

{
gant[g++] = p[i].id;

p[i].remaining --;

++;
executed = 1;

if (p[i].remaining <= 0)
completed++;

}

if (!executed) {
gantt[g++] = 0;
++;

}

printf("%n");

for (i=0; i<g; i++)
printf("----+");

printf("\n");

for (i=0; i<g; i++) {

if (gantt[i]==0)
printf("idle ");

else
printf("P%d", gantt[i]);

}

printf("\n");

for (i=0; i<g; i++)
printf("----+");

printf("\n");

for (i=0; i<g; i++)
printf("id", i);

return 0;

}

O/P

Enter number of process : 3

Enter AT, BT & weight for P1: 0

P2 : 0

4

P3 : 0

6

3

Enter time quantum : 6

P1	P2	P3	P3	P3	P1	P2	P3	P3	P3	P1	P1	P1	P1
1	2	3	4	5	6	7	8	9	10	11	12	13	14


```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < 10; i++) {
        int item = rand() % 100;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Producer produced: %d at index %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 0; i < 10; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        printf("Consumer consumed: %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
}
```

sleep(2);

column Null;

int main()

pthread_t prod, cons;

Sem_init(&empty, 0, BUFFER_SIZE);

Sem_init(&full, 0, 0);

pthread_create(&prod, NULL, producer, NULL);

pthread_create(&cons, NULL, consumer, NULL);

pthread_join(prod, NULL);

pthread_join(cons, NULL);

sem_destroy(&empty);

sem_destroy(&full);

pthread_mutex_destroy(&mutex);

return 0;

}

Output :

producer produced : 41 at index 0

consumer consumed : 41 from index 0

producer produced : 61 at index 1

C : 61 from index 1

P : 34 at index 2

P : 0 at index 3

C : 34 from index 2

P : 69 at index 4

P : 24 at index 0

C : 0 from index 3

P : 78 at index 1

P : 58 at index 2

C : 69 from index 4

P : 62 at index 3

P : 64 at index 4

C : 24 from index 0

C : 78 from index 1

C : 58 from index 2

C : 62 from index 3

C : 64 from index 4

Sp. 7
29/4/20